

TECHIN

# JavaScript įvadas

# Pavyzdžiai su JavaScript (JS)

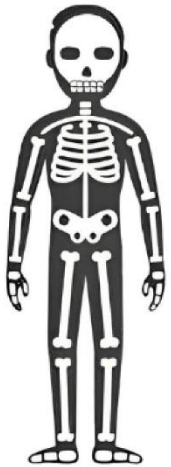
- ✓ Paspaudi mygtuką → Atsiranda pranešimas
- ✓ Paspaudi „Patinka“ → Padidėja  skaičius
- ✓ Pridedi prekę į krepšelį → Krepšelis atsinaujina
- ✓ Įvedi tekstą į kuriamo slaptažodžio laukelį → Patikrinama, ar slaptažodis pakankamai stiprus
- ✓ .....

**VARTOTOJAS → VEIKSMAS → PUSLAPIO REAKCIJA**

HTML



**HTML** the Skeleton



CSS



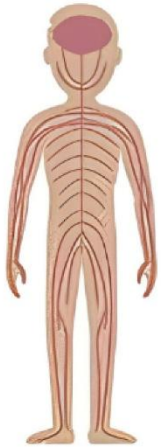
**CSS** the Skin

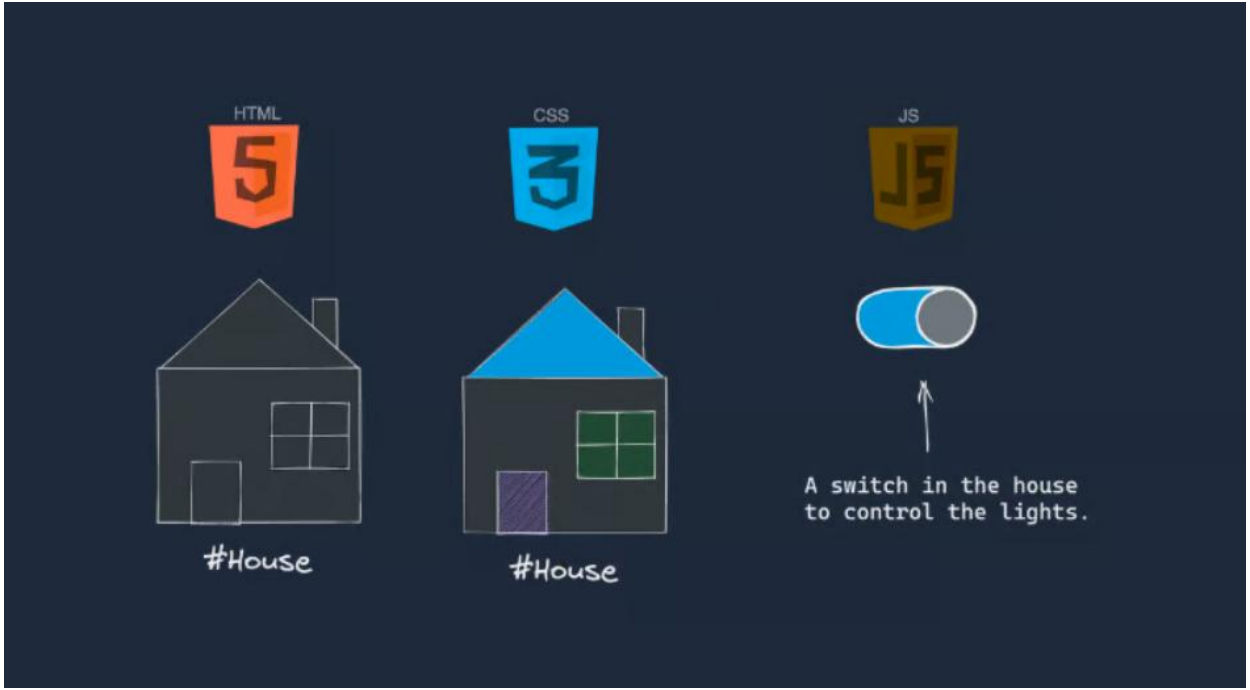


JS



**Javascript** the Brain






- **HTML** is used to build the basic structure of the web page.
- **CSS** is used to paint(design) that structure.
- **JavaScript** adds the behaviour to a web page.

# JavaScript (JS)

- Tai plačiai naudojama objektinė programavimo kalba.
- Sukūrė Brendan Eich 1995m. per 10 dienų (Netscape kompanija).
- Naudojama tiek frontend‘ui, tiek backend‘ui.



# JavaScript vs. Java

- **Tai visiškai skirtingos programavimo kalbos.**
- **Panašūs pavadinimai – marketingo strategija.**
- **Mocha → LiveScript → JavaScript**



# JavaScript versijos

| Ver | Official Name | Description |
|---|---|---|
| 1 | ECMAScript 1 (1997) | First Edition. |
| 2 | ECMAScript 2 (1998) | Editorial changes only. |
| 3 | ECMAScript 3 (1999) | Added Regular Expressions.<br>Added try/catch. |
| 4 | ECMAScript 4 | Never released. |
| 5 | ECMAScript 5 (2009)<br>Read More: JS ES5 | Added "strict mode".<br>Added JSON support.<br>Added String.trim().<br>Added Array.isArray().<br>Added Array Iteration Methods. |
| 5.1 | ECMAScript 5.1 (2011) | Editorial changes. |
| 6 | ECMAScript 2015<br>Read More: JS ES6 | Added let and const.<br>Added default parameter values.<br>Added Array.find().<br>Added Array.findIndex(). |
| 7 | ECMAScript 2016 | Added exponential operator (**).<br>Added Array.prototype.includes. |
| 8 | ECMAScript 2017 | Added string padding.<br>Added new Object properties.<br>Added Async functions.<br>Added Shared Memory. |
| 9 | ECMAScript 2018 | Added rest / spread properties.<br>Added Asynchronous iteration.<br>Added Promise.finally().<br>Additions to RegExp. |

**ECMAScript** = taisyklės
**JavaScript** = kalba, kuri tas taisykles įgyvendina



# JavaScript (JS)

- **JavaScript kodai yra vadinami skriptais ir gali būti rašomi tiesiai į HTML dokumentą ir pasileisti automatiškai puslapiui užsikrovus.**
- **Nėra kompiliuojami, vykdomi tiesiogiai kaip teksto eilutės.**

- **JS scriptai gali būti vykdomi:**
  - **Naršyklėje**
  - **Serveryje**
  - **Bet kokiame įrenginyje, kuris turi programą JavaScript engine-interpretatorių, skirtą JS kodui vykdyti**



# Ką gali JavaScript?

- **JavaScript nėra kompiliuojama, o interpretuojama programavimo kalba. T.y. interpretatorius vykdo kodą eilutę po eilutės, neversdamas jo į mašininį (baitinį) kodą.**
- **JavaScript galimybės priklauso nuo interpretatoriaus:**

**Browser Environment**

**Node.js Environment**

**(server environment)**



# Ką gali browser JavaScript?

- ✓ Sukurti naujus HTML elementus, pridėti ir pakeisti jų turinį.
- ✓ Pridėti/pakeisti CSS stilių.
- ✓ Reaguoti į pelės ir klaviatūros įvykius, vykdyti kodus įvykus pelės paspaudimui, užvedimui ant HTML elemento ir panašiai.
- ✓ Siųsti užklausas serveriams ir pasiimti iš jų duomenis.
- ✓ Dirbti su cookies. Užduoti lankytojams klausimus, rodyti pranešimus.
- ✓ Įsiminti duomenis kliento naršyklėje (local storage).



# JavaScript paleidimas su node.js

1. Sukurkite projekto katalogą
2. Pasitikrinkite, ar turimas node.js: terminale parašyti node --version
3. Suinstaliuoti node.js, jei neturite: https://nodejs.org/en
4. Atsidarykite projektą: pvz., kataloge pasileidus terminalą, jame parašykite code .
5. Projekto kataloge susikurkite, pvz., task1.js failą ir jame parašykite script‘ą console.log(“Hello World!!!”);
6. Atsidarykite terminalą VS Code ir parašykite script‘o iškvietimo komandą: node task1
7. Tekstas Hello World!!! turėtų būti matomas terminale.



# JavaScript prijungimas (Browser)

- ✓ Skriptas gali būti rašomas HTML dokumente, tarp žymių <head> arba prieš </body>
- ✓ Būtina rašyti žymes <script>

```
<!DOCTYPE html>
<html>
<body>

<h2>External JavaScript</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

<p>(myFunction is stored in an external file called
"myScript.js")</p>

<script src="myScript.js"></script>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
    <script src="/js/myScript.js"></script>
</head>
<body>

<h2>External JavaScript</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```



# JavaScript vykdymo atidėjimas

```
<body>
<script defer src="script.js"></script>
//HTML code
</body>
```

Atributas **defer** atideda script‘o vykdymą bei laukia, kol užsikraus HTML ir tik tada paleidžia script‘ą. Tai pagreitina puslapio užsikrovimą, ypač kai script‘ų vykdymas užtrunka.



# JavaScript prijungimas

## Be atributo **defer**

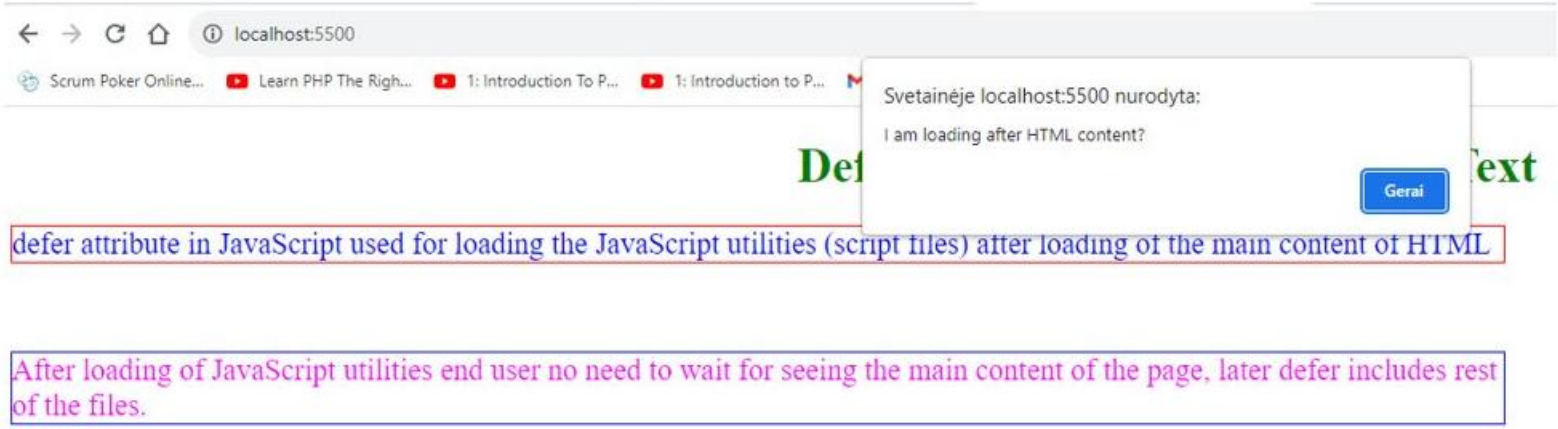
```
<body>
    <h1>Defer attribute with Paragraph Text</h1>
    <!--Including script file with defer attribute-->
    <script src="script.js"></script>
    <p class="p1">
        defer attribute in JavaScript used for loading the JavaScript utilities
        (script files) after loading of the main content of HTML</p>
    <br>
    <p class="p2">After loading of JavaScript utilities end user no need
        to wait for seeing the main content of the page, later defer includes
        rest of the files.</p>
</body>
```



# JavaScript prijungimas

## Su atributu **defer**

```
<body>
    <h1>Defer attribute with Paragraph Text</h1>
    <!--Including script file with defer attribute-->
    <script src="script.js" defer></script>
    <p class="p1">
        defer attribute in JavaScript used for loading the JavaScript utilities
        (script files) after loading of the main content of HTML</p>
    <br>
    <p class="p2">After loading of JavaScript utilities end user no need
        to wait for seeing the main content of the page, later defer includes
        rest of the files.</p>
</body>
```



# JavaScript sintaksė

**Kalba susideda iš sakinių.**
**Sakinius rekomenduojama atskirti kabliataškiais, nors tai nėra būtina.**

```
var a, b, c;     // Declare 3 variables
a = 5;           // Assign the value 5 to a
b = 6;           // Assign the value 6 to b
c = a + b;       // Assign the sum of a and b to c
```

**Komentarai rašomi:**
**//vienos eilutės**
**/* kelių eilučių*/**

**JavaScript kalba yra case sensitive.**



# JavaScript “use strict”

- Pirmoje *.js failo eilutėje rekomenduojama rašyti:

**“use strict” arba `use strict`**

- Tai griežtesnis kodo rašymo būdas.
- JavaScript kalba yra „loosely typed“.

- Sintaksės klaidas paverčia klaidų pranešimais (be “use strict“ klaidos gali būti nutylimos) – išvengiame nesuderinamumo ir kodo „lūžimo“.
- Kodas veikia greičiau ir yra saugesnis.



# JavaScript “use strict”

**JavaScript kalboje galime naudoti kintamuosius jų nedeklaravę:**

```
1  // note: no "use strict" in this example
2
3  num = 5; // the variable "num" is created if it didn't exist
4
5  alert(num); // 5
```

**Tai yra bloga praktika ir “use strict” mus nuo jos apsaugo:**

```
1  "use strict";
2
3  num = 5; // error: num is not defined
```



# JavaScript kintamieji

**Kintamasis yra duomenų saugojimo vieta, turinti vardą.**




**Kintamiesiems deklaruoti naudojami keyword‘ai:**

1. **let**
2. **const**
3. **var**

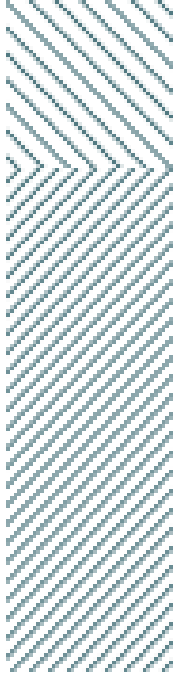
# JavaScript kintamieji
## let

**Norint sukurti (deklaruoti) kintamąjį, naudojamas keyword let.**

```
let message;
```

```
let message;
message = 'Hello!';

alert(message); // shows the variable content
```





# JavaScript kintamieji
## let

### Kintamųjų deklaravimas:

**Vienoje eilutėje:**

```
let user = 'John', age = 25, message = 'Hello';
```

**Keliose eilutėse:**

```
let user = 'John';
let age = 25;
let message = 'Hello';
```



# JavaScript kintamieji su let ir jų savybės

## Block scope (blokas {} ) arba Global scope:

```
let greeting = "say Hi";
let times = 4;

if (times > 3) {
     let hello = "say Hello instead";
     console.log(hello);//"say Hello instead"
 }
console.log(hello) // hello is not defined
```

# JavaScript kintamieji su let ir jų savybės

## Gali būti pakeista reikšmė, bet negali būti perrašytas tame pačiame scope:

```
let greeting = "say Hi";
greeting = "say Hello instead";
```

Ok

```
let greeting = "say Hi";
let greeting = "say Hello instead";//error: Identifier 'greeting' has already been declared
```

Error

```
let greeting = "say Hi";
if (true) {
    let greeting = "say Hello instead";
    console.log(greeting);//"say Hello instead"
}
console.log(greeting);//"say Hi"
```

Ok, nes skirtingi scope



# JavaScript kintamieji su let ir jų savybės

**Jei naudosime nedeklaruotą kintamąjį, t. y. deklaruosime vėliau nei naudosime, gausime Reference Error:**

```
greeter = "say hello";
let greeter;
```

```
▸ Uncaught ReferenceError: Cannot access 'greeter' before initialization
    at script.js:23
```



# JavaScript kintamieji su let ir jų savybės

The `let` keyword was introduced in ES6 (2015)

Variables declared with `let` have **Block Scope**

Variables declared with `let` must be **Declared** before use

Variables declared with `let` cannot be **Redeclared** in the same scope



# JavaScript kintamieji
## const

**Norint sukurti (deklaruoti) konstantą, naudojame keyword const:**

```
const myBirthday = '18.04.1982';
```

**const kintamieji negali būti perrašomi arba keičiama jų reikšmė:**

```
<script>
'use strict';

const myBirthday = '18.04.1982';

myBirthday = '01.01.2001'; // error, can't reassign the constant!
</script>
```



# JavaScript kintamieji
## const

**Konstantos negali būti perrašomos arba keičiama jų reikšmė.**

**Reikšmės keitimas:**

```
const greeting = "say Hi";
greeting = "say Hello instead";//error : Assignment to constant variable.
```

**Kintamojo greeting perrašymas:**

```
const greeting = "say Hi";
const greeting = "say Hello instead";//error : Identifier 'greeting' has already been declared
```



# JavaScript kintamieji
## const

**Konstantų vardai rašomi didžiosiomis raidėmis.**

**Pavyzdys, kaip galima panaudoti konstantas:**

```
const COLOR_RED = "#F00";
const COLOR_GREEN = "#0F0";
const COLOR_BLUE = "#00F";
const COLOR_ORANGE = "#FF7F00";

// ...when we need to pick a color
let color = COLOR_ORANGE;
alert(color); // #FF7F00
```



# JavaScript kintamieji

## var

**Tai „old school“ kintamojo deklaravimo būdas.**

```
var message = 'Hello';
```

**var gali būti global arba function scope. Jis neturi block scope.**

```
var tester = "hey hi";

function newFunction() {
    var hello = "hello";
}
console.log(hello); // error: hello is not defined
```

```
<script>
'use strict';

function sayHi() {
  if (true) {
    var phrase = "Hello";
  }

  alert(phrase); // works
}

sayHi();
alert(phrase); // Error: phrase is not defined (Check the Developer Console)
</script>
```

# JavaScript kintamieji

## var

## Gali būti perrašomi ir pakeičiama reikšmė.

**Kintamojo greeting perrašymas:**

```
var greeter = "hey hi";
var greeter = "say Hello instead";
```

OK

**Kintamojo reikšmės keitimas:**

```
var greeter = "hey hi";
greeter = "say Hello instead";
```

OK



# JavaScript kintamieji

## var

# Hoisting

**Tai JavaScript mechanizmas, kai visi kintamieji ir funkcijos automatiškai deklaruojami scope viršuje prieš pradedant vykdyti kodą.**

**Kodas:**

```
console.log (greeter);
var greeter = "say hello"
```

**Bus automatiškai interpretuojamas kaip:**

```
var greeter;
console.log(greeter); //greeter is undefined
greeter = "say hello"
```

**Klaidos nemes, kodas bus vykdomas toliau.**



# JavaScript kintamieji

## var

**Su var kyla tokia problema:**

```
var greeter = "hey hi";
var times = 4;

if (times > 3) {
    var greeter = "say Hello instead";
}

console.log(greeter) //"say Hello instead"
```

**Kintamojo greeter reikšmė perrašoma bloke if. Jeigu norėsime kintamąjį naudoti kituose kodo blokuose, kintamojo reikšmė gali tapti ne tokia, kokios tikėjotės.**

**Todėl geriau naudoti tik let ir const.**



# JavaScript kintamųjų vardai

**Kintamųjų vardams keliami reikalavimai:**

1. **Vardas gali būti sudarytas iš raidžių, skatmenų ir simbolių $ ir _**
2. **Pirmas vardo simbolis negali būti skaičius**
3. **Jeigu vardą sudaro keli žodžiai, naudojamas camelCase**
4. **Viduriniai brūkšniai, pvz. – , neleidžiami**
5. **apple ir AppLe yra du skirtingi kintamųjų vardai**
6. **Non-Latin raidės leidžiamos, bet nerekomenduojamos. Pvz.:**

```
let имя = '...';
let 我 = '...';
```

**Geroji praktika:** `var lunchCostPerStudent = 5;`

# JavaScript duomenų tipai

**Kintamieji JavaScript kalboje gali turėti bet kokią reikšmę ir kintamųjų tipas nėra iš anksto apibrėžiamas.**

```
// no error
let message = "hello";
message = 123456;
```

**JavaScript kalba yra „loosely typed“ arba „dynamically typed“ – duomenų tipai egzistuoja, bet kintamieji nepriskiriami jokiam duomenų tipui.**



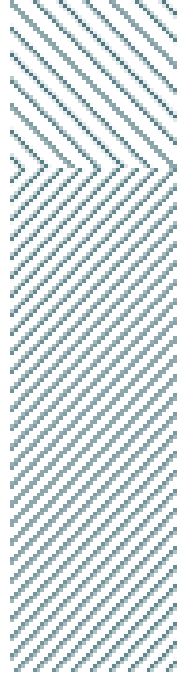
# JavaScript duomenų tipai

**Primityvai**

- Number (Infinity, Nan)
- BigInt
- String
- Boolean
- null
- Undefined
- Symbol – naudojamas objektų savybėms identifikuoti,

**Struktūriniai tipai**

- Objects: Arrays, Dates ...
- Functions
- String (gali būti ir objektas, priklauso nuo deklaracijos)





# JavaScript duomenų tipai

**Number – sveikieji skaičiai ir skaičiai su kableliu nuo $-2^{53}$ *iki* $2^{53}$**

```
1  let n = 123;
2  n = 12.345;
```

**Prie number taip pat priskiriama:**

- **Infinity – matematinė begalybė**

```
alert( 1 / 0 ); // Infinity
```

- **NaN (not a number) – neteisingo matematinio veiksmo rezultatas. Pvz.:**

```
alert( "not a number" / 2 ); // NaN, such division is erroneous
```



# JavaScript duomenų tipai

**BigInt – neriboto dydžio skaičiams**

```
1  // the "n" at the end means it's a BigInt
2  const bigInt = 12345678901234567890123456789012345678901234567890n;
```

**Prie skaičiaus prirašoma n raidė – tai reiškia, kad skaičius yra BigInt tipo.**



# JavaScript duomenų tipai

**String – tai tekstas. Reikšmės rašomos su kabutėmis.**

```
1  let str = "Hello";
2  let str2 = 'Single quotes are ok too';
3  let phrase = `can embed another ${str}`;
```

**JavaScript kalboje yra 3 rūšių kabutės:**

1. Double quotes: `"Hello"`.
2. Single quotes: `'Hello'`.
3. Backticks: `` `Hello` ``.



# JavaScript String `backticks`

- **Extended functionality quotes.**
- **Leidžia į tekstą įterpti kintamąjį arba reiškinį.**

```
1  let name = "John";
2
3  // embed a variable
4  alert( `Hello, ${name}!` ); // Hello, John!
5
6  // embed an expression
7  alert( `the result is ${1 + 2}` ); // the result is 3
```



# JavaScript String `backticks`

## Į tekstą gali būti rašomos ir HTML žymės:

```
const title = 'Best reads of 2019';
const author = 'Mario';
const likes = 30;
// creating html templates
let html = `
  <h2>${title}</h2>
  <p>By ${author}</p>
  <span>This blog has ${likes} likes</span>
`;
```

# JavaScript String

- String tipo duomenys tarp savęs ir su kintamaisiais gali būti jungiami ženklu +.
- Gali būti naudojamos viengubos arba dvigubos kabutės, taip pat backticks.

```
const title = 'Best reads of 2019';
const author = 'Mario';
const likes = 30;

// concatenation way
let result = 'The blog called ' + title + ' by ' + author + ' has ' + likes + ' likes';
```

# JavaScript duomenų tipai

**String – simboliai ir tekstai.**

- **Jeigu sukurti su kabutėmis (viengubomis, dvigubomis ir backticks) – tai yra primityvai.**
- **Jeigu sukurti su žodeliu new – tai objektai.**

```
let str1 = new String('What am I?');
typeof str1; // 'object'

let str2 = String('What am I?');
typeof str2; // 'string'

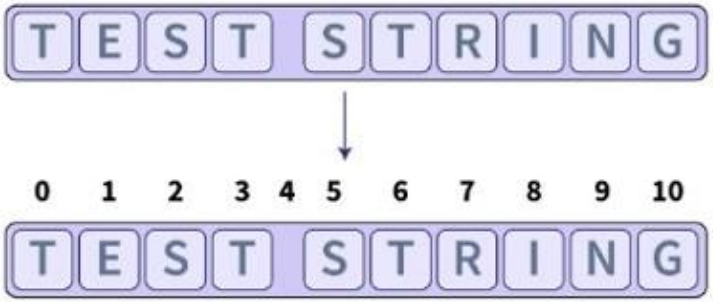
let str3 = "What am I?";
typeof str3; // 'string'
```

```
str1 === str2; // false
str1 === str3; // false
str2 === str3; // true
```



# JavaScript duomenų tipai

## String




```
let str = 'larch';
str[0]; // 'l'
str[1]; // 'a'
str[2]; // 'r'
str[3]; // 'c'
str[4]; // 'h'
```

```
str.charAt(0); // 'l'
```

# JavaScript duomenų tipai

## String primityvas yra immutable (nemutuojantis)

```
let str = 'larch';
typeof str; // 'string'
str[0] = 'm'; //This could throw an error if you are in strict mode
console.log(str); // 'larch'
```

## Bet kintamojo str reikšmę galima perrašyti:

```
let str = 'larch';
str = 'march'; // Reassigning str another value
console.log(str); // 'march'
```



# JavaScript duomenų tipai

## Boolean – tik dvi reikšmės: true arba false

```
let nameFieldChecked = true; // yes, name field is checked
let ageFieldChecked = false; // no, age field is not checked
```

```
let isGreater = 4 > 1;

alert( isGreater ); // true (the comparison result is "yes")
```



# JavaScript duomenų tipai

## null

**Tai speciali reikšmė, kuri reiškia „nieko“, „tuščia“ arba reikšmė nežinoma, bet ji yra priskirta.**

```
let age = null;
```

## undefined

**Reiškia „nepriskirta reikšmė“. Kintamasis deklaruotas, bet reikšmė nepriskirta, reiškia – reikšmė yra undefined.**

```
1  let x;
2
3  alert(x); // shows "undefined"
```



# JavaScript duomenų tipai

Non-zero value



0



null



undefined



# JavaScript aritmetiniai operatoriai

| Operator | Description |
|---|---|
| + | Addition |
| - | Subtraction |
| * | Multiplication |
| ** | Exponentiation (ES2016) (kėlimas laipsniu) pvz. 5**2 =25 |
| / | Division |
| % | Modulus (Division Remainder) |
| ++ | Increment |
| -- | Decrement |

https://www.w3schools.com/js/js_arithmetic.asp



# JavaScript priskyrimo operatoriai

| Operator | Example | Same As |
|---|---|---|
| = | x = y | x = y |
| += | x += y | x = x + y |
| -= | x -= y | x = x - y |
| *= | x *= y | x = x * y |
| /= | x /= y | x = x / y |
| %= | x %= y | x = x % y |
| **= | x **= y | x = x ** y |

https://www.w3schools.com/js/js_assignment.asp

